

Social Network System Design with PostgreSQL

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

A social network is defined by the graph between users (follows, friendships), and the content that flows along those edges (posts, likes, comments, shares). The primary technical challenge is the **feed generation problem**: given user A who follows 500 people, how do you generate a recent, relevant feed efficiently when those 500 people post hundreds of times per day? This document covers the full schema and the architectural tradeoffs between fan-out-on-write (push) and fan-out-on-read (pull) feed strategies.

Requirements

Functional Requirements

- User profiles with bio, avatar, follow counts
- Follow system (directed graph: A follows B; B does not necessarily follow A)
- Posts: text, images, videos, links
- Feed: chronological and algorithmic posts from followed users
- Likes on posts and comments
- Comments on posts (nested up to 2 levels)
- Hashtags and mentions (@username, #topic)
- Notifications: new follower, like, comment, mention
- Search: users, posts, hashtags
- Trending topics

Non-Functional Requirements

- 1B+ registered users
- 300M DAU
- 500M posts per day
- P99 feed load < 200ms
- P99 post creation < 100ms
- 99.99% uptime for feed reads
- Feed eventually consistent (seconds lag acceptable)

- Global content delivery

Capacity Estimation

Metric	Value
Registered users	1B
Daily active users	300M
Posts per day	500M
Likes per day	5B
Comments per day	500M
Follows per day	200M
Notifications per day	10B
Feed reads per day	6B (20 per DAU)

Storage Estimates

Table	Rows	Row Size	Total/Year
users	1B	1KB	1TB
posts	182B (lifetime)	800B	~73TB/yr
follows	500B (dense graph)	100B	50TB
likes	1.8T/yr	80B	144TB/yr
comments	182B/yr	500B	91TB/yr
notifications	3.65T/yr	200B	730TB/yr
feeds (push model)	1.5T/day × 365	80B	43PB/yr

Key observation: Fan-out-on-write for 1B users × 500M posts/day is infeasible at full scale. A hybrid approach is required.

Schema Design

```
-- =====
-- EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-osspl";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "btree_gin";

-- =====
```

```

-- ENUM TYPES
-- =====
CREATE TYPE post_type AS ENUM ('text', 'image', 'video', 'link', 'poll', 'story');
CREATE TYPE visibility AS ENUM ('public', 'followers', 'close_friends', 'private');
CREATE TYPE notification_type AS ENUM (
    'new_follower', 'post_liked', 'post_commented',
    'mentioned', 'reposted', 'followed_posted'
);

-- =====
-- USERS
-- =====
CREATE TABLE users (
    user_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    username         VARCHAR(50)   NOT NULL,
    display_name     VARCHAR(200)  NOT NULL,
    email            VARCHAR(320)  NOT NULL,
    password_hash    VARCHAR(255)  NOT NULL,
    bio              TEXT,
    avatar_url       VARCHAR(500),
    website          VARCHAR(500),
    location         VARCHAR(200),
    is_verified      BOOLEAN       NOT NULL DEFAULT FALSE,
    is_private       BOOLEAN       NOT NULL DEFAULT FALSE,
    -- Denormalized counts (updated via triggers/jobs)
    post_count       INT           NOT NULL DEFAULT 0,
    follower_count   INT           NOT NULL DEFAULT 0,
    following_count  INT           NOT NULL DEFAULT 0,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_username UNIQUE (username),
    CONSTRAINT uq_email  UNIQUE (email)
);

-- =====
-- FOLLOWS (Directed Graph)
-- =====
CREATE TABLE follows (
    follower_id      UUID          NOT NULL REFERENCES users(user_id) ON DELETE
    CASCADE,
    followee_id      UUID          NOT NULL REFERENCES users(user_id) ON DELETE
    CASCADE,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    is_close_friend  BOOLEAN       NOT NULL DEFAULT FALSE,
    -- For private accounts: 'pending' or 'accepted'
    status           VARCHAR(20)   NOT NULL DEFAULT 'accepted',
    PRIMARY KEY (follower_id, followee_id),
    CONSTRAINT chk_no_self_follow CHECK (follower_id != followee_id)
);

-- =====
-- POSTS

```

```

-- =====
CREATE TABLE posts (
    post_id          UUID          NOT NULL DEFAULT uuid_generate_v4(),
    author_id        UUID          NOT NULL REFERENCES users(user_id),
    post_type        post_type     NOT NULL DEFAULT 'text',
    body            TEXT,
    visibility        visibility   NOT NULL DEFAULT 'public',
    -- Repost support
    original_post_id UUID,          -- NULL = original post
    -- Engagement counts (denormalized)
    like_count       BIGINT        NOT NULL DEFAULT 0,
    comment_count    INT           NOT NULL DEFAULT 0,
    repost_count     INT           NOT NULL DEFAULT 0,
    view_count       BIGINT        NOT NULL DEFAULT 0,
    -- Metadata
    location         VARCHAR(300),
    is_deleted       BOOLEAN       NOT NULL DEFAULT FALSE,
    search_vector     TSVECTOR,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (post_id, created_at)
) PARTITION BY RANGE (created_at);

-- Monthly partitions
CREATE TABLE posts_2024_01 PARTITION OF posts
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
-- ...automate with pg_partman

-- Post media
CREATE TABLE post_media (
    media_id         UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    post_id          UUID          NOT NULL,
    created_at       TIMESTAMPTZ   NOT NULL,
    media_type       VARCHAR(20)   NOT NULL,    -- 'image','video','gif'
    url              VARCHAR(1000) NOT NULL,
    thumbnail_url    VARCHAR(1000),
    width            INT,
    height           INT,
    duration_secs    INT,
    sort_order       SMALLINT      NOT NULL DEFAULT 0
);

-- Hashtags
CREATE TABLE hashtags (
    tag_id           UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    name             VARCHAR(200)  NOT NULL,
    post_count       BIGINT        NOT NULL DEFAULT 0,
    CONSTRAINT uq_hashtag_name UNIQUE (lower(name))
);

CREATE TABLE post_hashtags (
    post_id          UUID          NOT NULL,

```

```

        created_at      TIMESTAMPTZ      NOT NULL,
        tag_id          UUID              NOT NULL REFERENCES hashtags(tag_id),
        PRIMARY KEY (post_id, tag_id)
    );

-- Mentions
CREATE TABLE post_mentions (
    post_id             UUID              NOT NULL,
    created_at          TIMESTAMPTZ      NOT NULL,
    mentioned_user_id   UUID              NOT NULL REFERENCES users(user_id),
    PRIMARY KEY (post_id, mentioned_user_id)
);

-- =====
-- LIKES
-- =====
CREATE TABLE likes (
    user_id             UUID              NOT NULL REFERENCES users(user_id) ON DELETE
CASCADE,
    post_id             UUID              NOT NULL,
    post_created_at     TIMESTAMPTZ      NOT NULL,
    liked_at            TIMESTAMPTZ      NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, post_id)
) PARTITION BY RANGE (liked_at);

CREATE TABLE likes_2024_01 PARTITION OF likes
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- =====
-- COMMENTS
-- =====
CREATE TABLE comments (
    comment_id          UUID              NOT NULL DEFAULT uuid_generate_v4(),
    post_id             UUID              NOT NULL,
    post_created_at     TIMESTAMPTZ      NOT NULL,
    author_id           UUID              NOT NULL REFERENCES users(user_id),
    parent_comment_id   UUID,            -- NULL = top-level, non-NULL = reply
    body                TEXT              NOT NULL,
    like_count          INT               NOT NULL DEFAULT 0,
    is_deleted          BOOLEAN           NOT NULL DEFAULT FALSE,
    created_at          TIMESTAMPTZ      NOT NULL DEFAULT now(),
    PRIMARY KEY (comment_id, created_at)
) PARTITION BY RANGE (created_at);

-- =====
-- FEED (Fan-out-on-write for regular users)
-- =====
-- Feed entries are pre-computed for users with < 10K followers (fanout is
practical)
-- Users with > 10K followers (celebrities): feed is computed on-read (pull)
CREATE TABLE user_feeds (
    user_id             UUID              NOT NULL,

```

```

    post_id          UUID          NOT NULL,
    post_created_at  TIMESTAMPTZ    NOT NULL,
    author_id        UUID          NOT NULL,
    score            FLOAT,         -- for algorithmic feed; NULL = chronological
    inserted_at      TIMESTAMPTZ    NOT NULL DEFAULT now(),
    PRIMARY KEY (user_id, post_id)
) PARTITION BY RANGE (inserted_at);

-- Hot feed: last 3 days in one partition (fits in buffer cache)
CREATE TABLE user_feeds_hot PARTITION OF user_feeds
    FOR VALUES FROM (now() - INTERVAL '3 days') TO ('9999-01-01');

-- =====
-- NOTIFICATIONS
-- =====
CREATE TABLE notifications (
    notification_id UUID          NOT NULL DEFAULT uuid_generate_v4(),
    user_id          UUID          NOT NULL REFERENCES users(user_id),
    notification_type notification_type NOT NULL,
    actor_id         UUID          REFERENCES users(user_id),
    post_id          UUID,
    is_read          BOOLEAN       NOT NULL DEFAULT FALSE,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (notification_id, created_at)
) PARTITION BY RANGE (created_at);

-- =====
-- TRENDING TOPICS
-- =====
-- Pre-computed by a background job every 15 minutes
CREATE TABLE trending_topics (
    tag_id           UUID          NOT NULL REFERENCES hashtags(tag_id),
    country_code     CHAR(2),      -- NULL = global
    rank             SMALLINT      NOT NULL,
    post_count_1h    INT           NOT NULL,
    post_count_24h   INT           NOT NULL,
    velocity         FLOAT,        -- rate of change
    computed_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (country_code, rank)
);

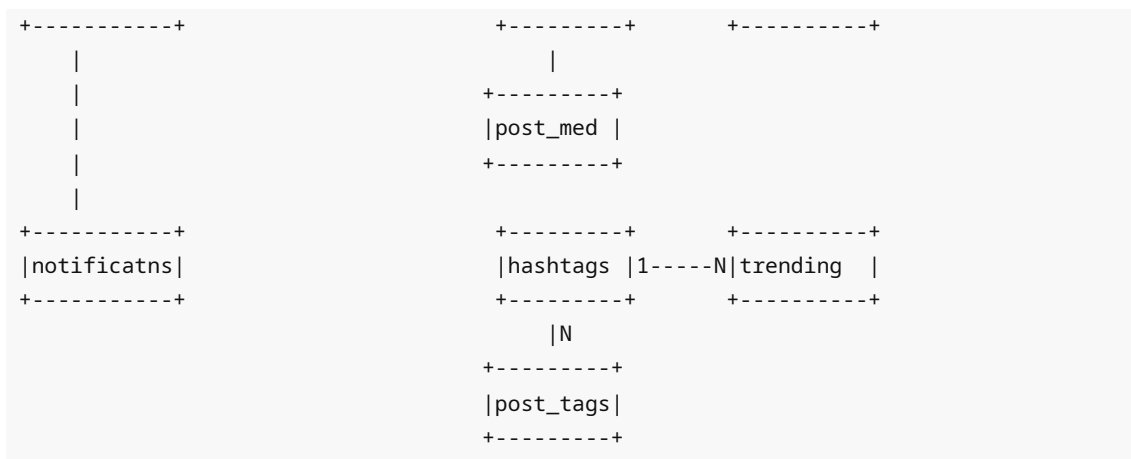
```

ASCII ER Diagram

```

+-----+      +-----+      +-----+
| users |N-----M| follows |      | posts |
+-----+      +-----+      +-----+
    |1                      |1
    |N                      |N
+-----+      +-----+      +-----+
|user_feeds |      | likes |      |comments |

```



Indexing Strategy

```

-- Users: search and auth
CREATE INDEX idx_users_username ON users (lower(username));
CREATE INDEX idx_users_username_trgm ON users USING GIN (username gin_trgm_ops);
CREATE INDEX idx_users_email ON users (lower(email));

-- Follows: bidirectional lookup
-- Primary key covers (follower_id, followee_id)
CREATE INDEX idx_follows_followee ON follows (followee_id, follower_id)
WHERE status = 'accepted';

-- Posts: author timeline
CREATE INDEX idx_posts_author ON posts (author_id, created_at DESC)
WHERE NOT is_deleted;

-- Posts: full-text search
CREATE INDEX idx_posts_search ON posts USING GIN (search_vector)
WHERE NOT is_deleted;

-- Posts: hashtag timeline
CREATE INDEX idx_post_hashtags_tag ON post_hashtags (tag_id, created_at DESC);

-- Likes: who liked a post (for like list display)
CREATE INDEX idx_likes_post ON likes (post_id, liked_at DESC);

-- Likes: did current user like this post (hot path)
-- Primary key (user_id, post_id) covers this already

-- Comments: top-level comments per post
CREATE INDEX idx_comments_post ON comments (post_id, created_at ASC)
WHERE parent_comment_id IS NULL AND NOT is_deleted;

-- Comments: replies to a comment
CREATE INDEX idx_comments_parent ON comments (parent_comment_id, created_at ASC)
WHERE parent_comment_id IS NOT NULL;

```

```
-- User feeds: personalized timeline (hot read)
CREATE INDEX idx_feeds_user ON user_feeds (user_id, post_created_at DESC);

-- Notifications: unread inbox
CREATE INDEX idx_notifs_user ON notifications (user_id, created_at DESC,
is_read);
```

Query Patterns

Q1 — User Profile Page

```
SELECT
  u.user_id, u.username, u.display_name,
  u.bio, u.avatar_url, u.website,
  u.is_verified, u.is_private,
  u.post_count, u.follower_count, u.following_count,
  -- Follow status of current viewer
  f.status AS follow_status
FROM users u
LEFT JOIN follows f
  ON f.follower_id = $current_user_id AND f.followee_id = u.user_id
WHERE u.username = lower($username);
```

Q2 — Home Feed (Fan-out-on-write path)

```
-- Pull from pre-computed user_feeds table
SELECT
  p.post_id, p.created_at, p.body, p.post_type,
  p.like_count, p.comment_count, p.repost_count,
  u.user_id AS author_id, u.username, u.display_name, u.avatar_url, u.is_verified,
  -- Did the viewer like this post?
  EXISTS (
    SELECT 1 FROM likes l
    WHERE l.post_id = p.post_id AND l.user_id = $current_user_id
  ) AS liked_by_me
FROM user_feeds uf
JOIN posts p ON p.post_id = uf.post_id AND p.created_at = uf.post_created_at
JOIN users u ON u.user_id = p.author_id
WHERE uf.user_id = $current_user_id
  AND uf.inserted_at >= now() - INTERVAL '7 days'
  AND NOT p.is_deleted
ORDER BY uf.post_created_at DESC
LIMIT 20 OFFSET $offset;
```

Q3 — Fan-out on Post Creation (background job)


```

-- Insert into every follower's feed
INSERT INTO user_feeds (user_id, post_id, post_created_at, author_id)
SELECT
    f.follower_id,
    $post_id,
    $post_created_at,
    $author_id
FROM follows f
WHERE f.followee_id = $author_id
    AND f.status = 'accepted'
    -- Skip users who haven't been active in 30 days (optimization)
    AND EXISTS (
        SELECT 1 FROM users u
        WHERE u.user_id = f.follower_id
            AND u.updated_at > now() - INTERVAL '30 days'
    )
ON CONFLICT DO NOTHING;
-- This runs in batches for authors with many followers

```

Q4 — Like a Post (Toggle)

```

BEGIN;

-- Try to insert (like)
INSERT INTO likes (user_id, post_id, post_created_at, liked_at)
VALUES ($user_id, $post_id, $post_created_at, now())
ON CONFLICT (user_id, post_id) DO NOTHING
RETURNING 'liked' AS action;

-- If already liked, delete (unlike) – use the returned action to decide
-- In two-step approach:
-- Step 1: INSERT ... ON CONFLICT DO NOTHING → check rowcount
-- Step 2: if 0 rows, DELETE FROM likes WHERE user_id = $1 AND post_id = $2

-- Update post counter
UPDATE posts
SET like_count = like_count + $delta    -- +1 for like, -1 for unlike
WHERE post_id = $post_id AND created_at = $post_created_at;

COMMIT;

```

Q5 — Post Comments Thread

```

SELECT
    c.comment_id, c.author_id, c.body, c.created_at,
    c.like_count, c.parent_comment_id,
    u.username, u.display_name, u.avatar_url,
    -- Replies count
    (SELECT COUNT(*) FROM comments r

```

```

        WHERE r.parent_comment_id = c.comment_id
        AND NOT r.is_deleted) AS reply_count
FROM comments c
JOIN users u ON u.user_id = c.author_id
WHERE c.post_id = $post_id
    AND c.parent_comment_id IS NULL
    AND NOT c.is_deleted
ORDER BY c.created_at ASC
LIMIT 20 OFFSET $offset;

```

Q6 — Hashtag Feed

```

SELECT
    p.post_id, p.created_at, p.body, p.post_type,
    p.like_count, p.comment_count,
    u.username, u.display_name, u.avatar_url
FROM post_hashtags ph
JOIN posts p ON p.post_id = ph.post_id AND p.created_at = ph.created_at
JOIN users u ON u.user_id = p.author_id
WHERE ph.tag_id = $tag_id
    AND p.created_at < $before_cursor
    AND NOT p.is_deleted
    AND p.visibility = 'public'
ORDER BY p.created_at DESC
LIMIT 30;

```

Q7 — User Search

```

SELECT
    u.user_id, u.username, u.display_name,
    u.avatar_url, u.is_verified, u.follower_count,
    similarity(u.username, $query) AS sim
FROM users u
WHERE u.username % $query -- trigram match (pg_trgm)
    OR u.display_name ILIKE '%' || $query || '%'
ORDER BY u.is_verified DESC, u.follower_count DESC, sim DESC
LIMIT 20;

```

Q8 — Trending Topics

```

SELECT
    h.name AS hashtag,
    t.post_count_1h,
    t.post_count_24h,
    t.velocity
FROM trending_topics t
JOIN hashtags h ON h.tag_id = t.tag_id
WHERE t.country_code = $country_code OR t.country_code IS NULL

```

```
ORDER BY t.rank ASC
LIMIT 10;
```

Q9 — Notification Feed

```
SELECT
    n.notification_id, n.notification_type,
    n.is_read, n.created_at,
    u.username AS actor_username,
    u.avatar_url AS actor_avatar,
    p.body AS post_preview
FROM notifications n
LEFT JOIN users u ON u.user_id = n.actor_id
LEFT JOIN posts p ON p.post_id = n.post_id
WHERE n.user_id = $current_user_id
    AND n.created_at >= now() - INTERVAL '30 days'
ORDER BY n.created_at DESC
LIMIT 50;
```

Q10 — Author Post Timeline

```
SELECT
    p.post_id, p.created_at, p.body, p.post_type,
    p.like_count, p.comment_count, p.repost_count,
    json_agg(pm.url ORDER BY pm.sort_order) FILTER (WHERE pm.media_id IS NOT NULL)
AS media_urls
FROM posts p
LEFT JOIN post_media pm ON pm.post_id = p.post_id AND pm.created_at = p.created_at
WHERE p.author_id = $1
    AND NOT p.is_deleted
    AND p.created_at < $before_cursor
    AND p.visibility IN ('public', 'followers') -- based on viewer's relationship
GROUP BY p.post_id, p.created_at
ORDER BY p.created_at DESC
LIMIT 20;
```

Scaling Strategy

The Feed Problem

This is the core scaling challenge of every social network:

Option A — Fan-out-on-write (push model)

- When user A posts, insert into every follower's feed table
- Pros: Feed reads are instant (simple SELECT on user_feeds)
- Cons: High-follower accounts (celebrities) cause write amplification (1 post → 10M feed inserts)

Option B — Fan-out-on-read (pull model)

- No pre-computed feeds; query follows graph at read time
- Pros: No write amplification; always current
- Cons: `JOIN follows ON posts WHERE followed_users IN (...)` is expensive at 500 follows

Option C — Hybrid (industry standard)

- Regular users (< 10K followers): fan-out-on-write
- Celebrities (> 10K followers): fan-out-on-read (cached)
- Feed is: `SELECT FROM user_feeds UNION ALL SELECT FROM recent_celebrity_posts WHERE followed`

Phase 1: Single Database (0–50M users)

- PostgreSQL with read replicas
- Feed from `user_feeds` table (fan-out on write via background job)
- Redis cache for hot feeds and notification counts
- Posts partitioned monthly

Phase 2: Service Decomposition (50M–300M users)

- UserService, PostService, FeedService, NotificationService — separate DBs
- FeedService: Redis-primary (`ZSET user:{id}:feed` with score = timestamp)
- Fan-out: Kafka topic `post.created` → FeedWorker → inserts into Redis ZSETs
- PostgreSQL: source of truth for posts and follows; not in feed read path

Phase 3: Horizontal Scaling (300M–1B users)

- UserDB sharded by `hash(user_id) % N`
- PostDB sharded by `hash(author_id) % N`
- Feed entirely in Redis cluster (not PostgreSQL) — 7-day window
- Older feed scrolling: PostgreSQL pull query (falls off hot path)
- Notifications: Cassandra (optimized for time-series fan-out writes)
- Trending: Redis SortedSet (`ZINCRBY trending:global hashtag_id 1` on every post)

Tradeoffs

Decision	Chosen	Alternative	Why
Fan-out-on-write for feed	Hybrid (write for regular, read for celebrities)	Pure pull	Pure pull can't serve 500M DAU at <200ms; write fan-out to celebrities is infeasible
Denormalized like/comment counts	Stored counter on posts	COUNT(*) queries	COUNT(*) on billions of likes per post page load is unacceptable
Monthly post partitions	Monthly	Weekly	Posts are large; monthly balances manageability with performance
GIN trgm index for user search	pg_trgm similarity	Full Elasticsearch	For user search (by username), trigram is sufficient and avoids a separate service
Soft delete posts	is_deleted = TRUE	Hard DELETE	Preserves notifications, likes, comments that reference the post;

			recovery possible
--	--	--	-------------------

Interview Discussion Points

- 1. How does the feed work for a new follower?** When A starts following B, A's feed doesn't retroactively fill with B's old posts (cold-start is empty for B). New posts from B are fanned out to A going forward. To see B's historical posts, A visits B's profile timeline (separate query).
- 2. How do you handle the celebrity problem (10M+ followers)?** Celebrities are tagged (`is_celebrity = TRUE` when `follower_count > threshold`). When they post, no fan-out occurs. Instead, when a user loads their feed, a separate query fetches recent celebrity posts from followed celebrities and merges them into the Redis ZSET result.
- 3. How are trending topics computed?** A background job runs every 15 minutes: `SELECT tag_id, COUNT(*) FROM post_hashtags WHERE created_at > now() - INTERVAL '1 hour' GROUP BY tag_id ORDER BY COUNT(*) DESC`. It compares to the 24-hour baseline to compute `velocity` (acceleration of posts). Results are written to `trending_topics` and cached in Redis.
- 4. How do you prevent the thundering herd on a viral post?** A post going viral generates millions of concurrent like/comment writes on the same row. Use Redis counters for real-time counts (`INCR post:{id}:likes`). Sync to PostgreSQL in batches every 60 seconds. The PostgreSQL `like_count` column is eventually consistent (± 60 seconds lag).

Common Interview Follow-ups

Q: How do you implement the "liked by friends" feature? A: `SELECT u.username FROM likes l JOIN follows f ON f.followee_id = l.user_id AND f.follower_id = $current_user WHERE l.post_id = $post_id LIMIT 3`. Returns up to 3 mutual connections who liked the post.

Q: How would you implement an algorithmic feed (not just chronological)? A: The `user_feeds.score` column stores an ML-computed relevance score. The ML model runs offline, considering: recency, interaction rate, relationship strength (close_friend vs. casual follow), content type preference. Updated scores are batch-written to `user_feeds`. Feed queries order by `score DESC` instead of `post_created_at DESC`.

Q: How do you handle blocked users? A: Add a `blocks(blocker_id, blocked_id)` table. Filter out blocked users in all queries: feed, comments, search. In feed fanout, skip inserting into feeds where the author is blocked by the recipient. In pull queries, add `AND NOT EXISTS (SELECT 1 FROM blocks WHERE blocker_id = $viewer AND blocked_id = p.author_id)`.

Performance Considerations

- Feed read is the hottest path:** `user_feeds` is read ~ 20 times per DAU $\times 300M = 6B$ reads/day. This table must be in Redis (ZSET) for production scale. PostgreSQL `user_feeds` is the fallback and source of truth.
- Like counter hot rows:** Posts going viral receive thousands of likes/second on the same row. Use Redis `INCR` + batch write to PostgreSQL every 60s. Otherwise, row-level lock contention on `like_count` becomes a bottleneck.
- Partition pruning on posts:** `created_at` is both the partition key and the primary sort. All feed queries include `created_at < $cursor` to enable pruning.

- **Trigram index size:** GIN trgm index on `username` is $\sim 3\times$ the column size. For 1B users, this is $\sim 150\text{GB}$. Consider keeping only on a read replica dedicated to search.
 - **Connection pooling for burst:** Viral content causes traffic spikes ($10\times$ normal). PgBouncer absorbs spikes; pool size 200 connections serves thousands of concurrent clients.
-

Cross-References

- See `18_Architecture_Case_Studies/06_messaging_system.md` for LISTEN/NOTIFY and notification delivery
- See `07_analytics_platform_design.md` for engagement analytics pipeline
- See `08_system_design_framework.md` for interview presentation framework
- PostgreSQL full-text search: Chapter 9 of this guide
- Partitioning: Chapter 5 of this guide
- JSONB and GIN indexes: Chapter 7 of this guide